1. The "Zombie" Container (Exits Immediately)

Symptom: You run docker run, but docker ps shows nothing. docker ps -a shows the container as Exited (0) or Exited (1)

Cause: A Docker container only stays alive as long as its foreground process is running. If your script finishes or your service (like Nginx) starts in "daemon" mode (background), the container thinks its job is done and shuts down.

The Fix: Ensure your entrypoint script ends with a long-running process.

Use tail -f /dev/null as a temporary hack to keep it alive during debugging.

Interview Answer: "I check the logs using docker logs <id> to see if it crashed or simply finished its task."

2. "It Works Locally, Fails in Container" (Networking)

Symptom: Your app can't connect to the database, even though they are both running in Docker.

Cause: Using localhost inside a container refers to that specific container, not your computer.

The Fix: Put both containers on the same User-Defined Bridge Network.

Connect using the container name as the hostname (e.g., mongodb://db-container:27017).

3. The "Silent Killer": OOM (Out of Memory)

Symptom: A container is running fine and suddenly disappears or restarts without a specific error message in the logs.

Cause: The container exceeded its memory limit (or the host's memory), and the Linux kernel triggered the OOM Killer to save the system.

The Fix: Check for Exit Code 137 (the universal sign for OOM).

Use docker stats to monitor real-time usage.

Set explicit limits: docker run --memory="512m" ...

© 2026 corporealsoul. All rights reserved.

4. Permission Denied (Volumes & Bind Mounts)

Symptom: Your app crashes because it "cannot write to folder /data" or "cannot find config.yaml."

Cause: The user inside the container (often root or a low-privilege node/python user) doesn't have the same UID/GID as the user on the host machine.

The Fix: Use chown on the host folder to match the container's user ID.

Pass the user ID at runtime: docker run -u $(id -u):$(id -g).

5. Image Bloat (Slow Build/Deploy)

Symptom: Your Docker image is 2GB+ and takes forever to push to the registry.

Cause: Including build tools, cache files, and source code that isn't needed for production.

The Fix: Multi-stage builds: Use one stage for building and a "slim" stage (like alpine or distroless) for the final image.

Use a .dockerignore file to skip node_modules or .git folders.

Quick Troubleshooting Table

Exit Code	Meaning	Immediate Action
0	Clean Exit	Check if the process was supposed to stay in foreground.
1	Application Crash	Check docker logs for code errors.
127	Command Not Found	Check your PATH or if the binary exists in the image.
137	OOMKilled	Increase memory limits or check for memory leaks.
139	Segfault	Binary incompatibility (common with Alpine vs Debian).

© 2026 corporealsoul. All rights reserved.